ML-based Data Matrix image localization and segmentation for standard scanners

Supervisors: Yucheng Lu (80%) & Veronika Cheplygina (20%)
In collaboration with MAN Energy Solutions

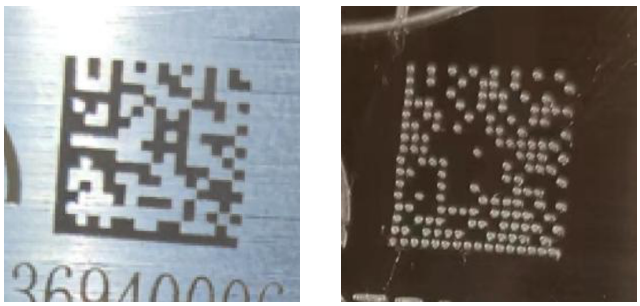
Aidan Stocks — aist@itu.dk

***Abstract*—Data Matrix codes are used at MAN Energy Solutions to serialize various machined metal components. Historically, decoding these codes has been done using algorithmic approaches without machine learning, such as in the case of the open-source library libdmtx. While libdmtx performs well under typical conditions, it often fails in industrial environments such as factories and ships, where poor lighting and damaged codes are common. To address these challenges, this research leverages recent advances in object detection and image segmentation, employing a cascaded model integrating YOLOv11 and a modified U-Net architecture to improve the performance of libdmtx. Experimental results demonstrate a 12% increase in decode rate under these industrial conditions, though at the expense of processing time, highlighting opportunities for further optimizing these methods for real-time applications.**

***Index Terms*—Data Matrix, YOLO, Object Detection, U-Net, Image Binarization, Segmentation**

I. Introduction

In industrial engineering settings, Data Matrix codes (DMCs) are often marked directly on machined metal components to serialize them. MAN Energy Solutions requires manufacturers to use DMCs to serialize engine components so that they are verifiably from a MAN-approved supplier. Although DMC scanners work in ideal conditions, they often fail when scanning from smartphone cameras while on board ships or in specific factory settings. This is usually due to poor quality cameras, the DMCs being physically small, on reflective (metal) surfaces, or due to poor lighting conditions. Most scanners are also not designed to recognize dot-peen marked Data Matrix codes, a commonly used and cheaper alternative method for marking components seen in Figure 1b. This paper investigates how machine learning methods can improve the speed and decode rate of the existing open-source DMC scanner library libdmtx[1] in this industrial setting.



(a) Laser **(b)** Dot Peen

Fig. 1: Difference between a laser and dot peen marked DMC.

A two-step cascaded model implementing YOLO[2] object detection to localize the DMC, and U-Net[3] segmentation for image binarization to remove noise, successfully improves the decode rate of the standard scanner at the cost of increased runtime.

By preprocessing the image with YOLO cropping and a U-Net-based image binarization, some steps of libdmtx are effectively done for it: YOLO for efficient localization and U-Net for robust image binarization.

This research paper focused on using models that solve the tasks well and fast, ideally in real-time, where an input image can be processed at speeds greater than one frame per second (> 1 FPS). Previous works [4]–[6] show that object detection models such as YOLO[2] can perform object detection tasks accurately and in real-time speed. The lightest architecture of one of the most recent YOLO models developed by Ultralytics, yolo11n[7], was chosen for the localization task due to its improved accuracy and speed over previous versions.

Other works[8], [9] show that adaptive threshold binarization is an image-processing technique effective in preprocessing QR codes for the decoding process. One paper[10] shows that a more complex image processing model such as U-Net can binarize images better than methods that calculate global binarization thresholds[11] in highly noisy images with adverse lighting conditions. Therefore, we propose using the U-Net[3] architecture with an attached sigmoid and binarization head thresholded at 0.5.

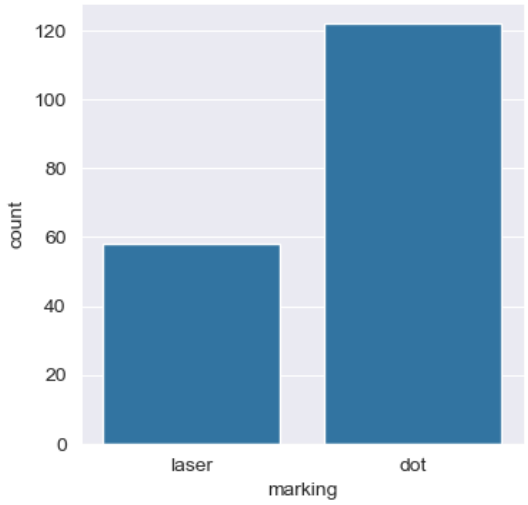
The code used to produce these results, along with a README of the exploratory process, is available at this GitHub repository.

II. Data Gathering

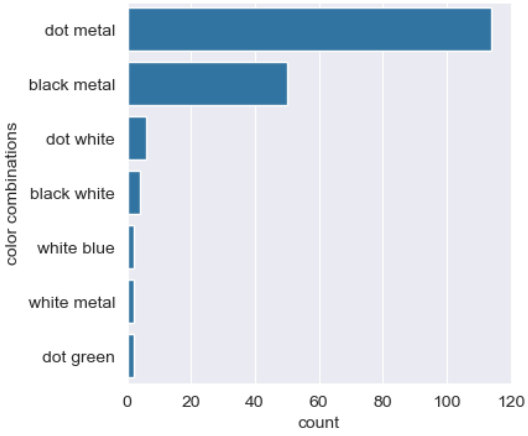
For training and testing the models, a dataset containing images of both laser and dot-peen-marked metal components was provided by MAN Energy Solutions. It contains 180 images, the distributions of which can be seen in Figure 2.

Most images contain DMCs made by dot-peen marking, and most have different types of distortions on their DMCs. A sample of the images manually cropped to their DMCs is shown in Figure 3 for convenience.

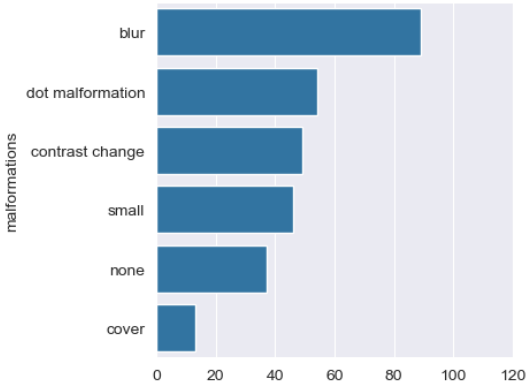
Only 50 of the 180 images were kept as a testing dataset to allow room for fine-tuning models.



(a) Counts of Laser and Dot Peen marked DMCs. Note the high amount of dot-peen marked components.

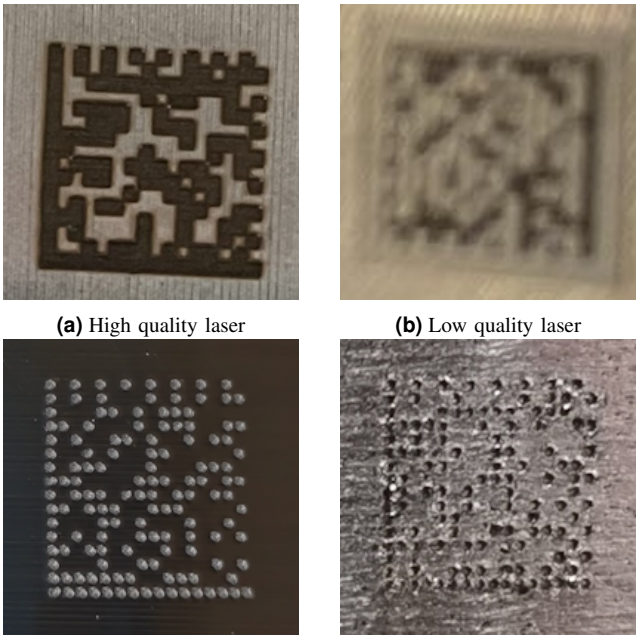


(b) Counts of combinations of DMC (left) and background (right) colors. Dot-peen marked components are marked "dot". Most laser-marked components are black on metal (lasered directly onto the metal), but cases of markings on white stickers and other materials also exist. In most cases, the background color is metallic, while the DMC is either a hole from dot-peen marking or blackened metal from a laser.



(c) Counts of malformation type present

Fig. 2: Difference between a laser and dot peen marked DMC. Note the high amount of blur and contrast change in all images and the high amount of malformations in the dot-peen marked components. "Contrast change" denotes a variation in background color due to light reflections, causing a difference in contrasting colors between DMC and background.



(a) High quality laser **(b)** Low quality laser

(c) High quality dot peen **(d)** Low quality dot peen

Fig. 3: Examples of differing quality between DMCs.

Roboflow[12] was used for train validation test splitting into sizes of 100, 30, and 50 respectively. Roboflow was also used to augment the training set and increase its size. The following pre-processing and augmentation steps were performed:

1) Auto orientation.
2) Resize to 640x640 (input size for yolo11n)
3) Rotations of 90°, 180°, and 270°.
4) Shearing of ± 5 in both horizontal and vertical dimensions.

The augmented dataset can be found on Roboflow here.

III. Models

A. Framework Overview

Condensed, libdmtx uses the following steps to decode a given image containing a DMC:

1) Image Preprocessing. The input image is grayscaled and binarized with a specified or calculated threshold to help reduce noise.
2) Localization. The DMC finder pattern ("L" shape) is localized, giving an approximate area of the image that contains the DMC.
3) Geometric Alignment. The DMC orientation is corrected, and a grid is overlayed on the DMC.
4) Symbol Decoding. The grid is iterated over, and the contrast between the black pixels of the DMC and the white pixels of the background is used to convert the DMC into encoded bits. Reed-Solomon error correction is used.
5) Data Extraction. The bits are finally decoded into the desired string or binary data.

The first two steps of the libdmtx decoder, image binarization & DMC area localization, are effectively replaced by

the two cascaded models. However, we perform the two steps in reverse order because YOLO is a computationally cheaper model architecture to run than U-Net. By doing the steps in reverse, YOLO's speed can be utilized to detect and crop to the DMC area relatively fast, leading to fewer pixels for the slower U-Net model to process for binarization.

Concretely, we end up with a framework where YOLO is used to crop an image down its DMC, U-Net is used to segment and binarize the DMC from the rest of the cropped image, and libdmtx is used to decode the binarized DMC.

B. DMC Localization

During the early exploration phase, the Ultralytics yolo11n pre-trained model was tested on all MAN images to see if any of the 80 classes from the dataset it was trained on (COCO[13]) would identify the DMCs. Only two images produced correct detections, proving that the YOLO model needed to be retrained or finetuned for the domain. The two detections can be seen in Figure 4.

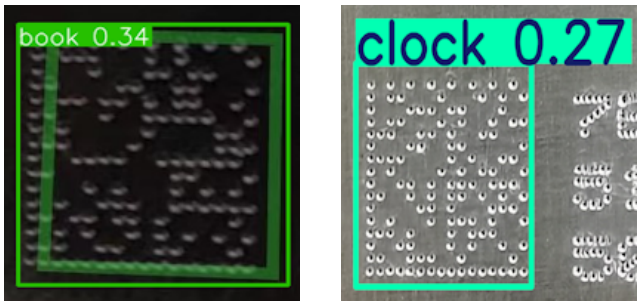


Fig. 4: Valid DMC detections from pretrained yolo11n.

Roboflow[12] was used for manual annotation of the dataset, drawing the bounding boxes to calculate and save the required x, y, w, and h of them. The (x, y) coordinates denote the center of the predicted bounding box, while the (w, h) denote the width and height of the box. YOLO models for object detection tasks predict these values.

Three different YOLO models were trained and evaluated.

1) YOLO from Scratch: A dataset publicly available on Kaggle[14] was used to train a YOLO model from scratch. The dataset contains images of QR codes both on plain white backgrounds and surrounded by text. The dataset also supplies the associated YOLO bounding box annotations for each image. Figure 5 shows samples of both types of images present in the Kaggle dataset.

A YOLO model trained from scratch on this dataset was initially considered because of the visual similarities between the two code types, potentially allowing for effective transfer learning. Figure 6 from another paper[15] shows the similarities and differences between the two code types.

2) Kaggle - Finetuned: Kaggle Scratch was finetuned on the MAN training and validation sets to produce another model for comparison. By fine-tuning to the real-world images, the model's performance should improve.

3) YOLO - Finetuned: Following Ultralytics recommendations, a final model for comparison is produced by fine-tuning their pre-trained yolo11n model on the MAN training and validation sets.

(a) Simple **(b)** Complex

Fig. 5: Examples from Kaggle dataset.

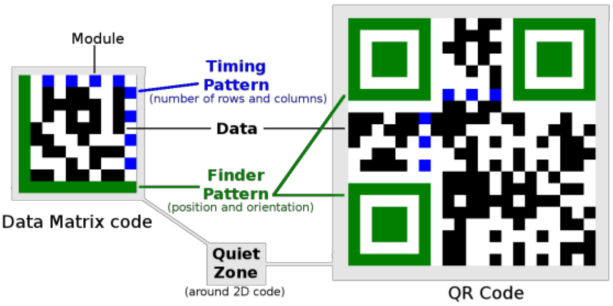


Fig. 6: Comparison between Data Matrix code and QR code. Note the main difference in finder and timing patterns.

C. DMC Binarization

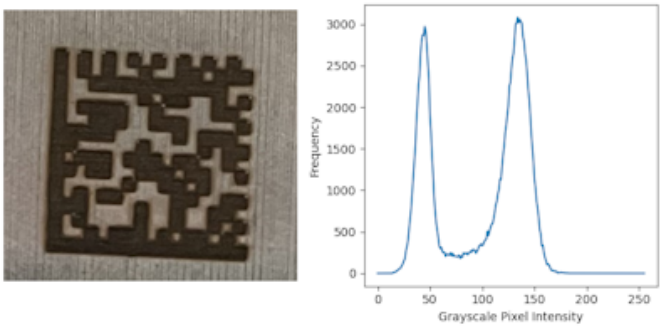
During the exploration phase of this paper, methods other than U-Net were tried and failed.

First, we trained a ResNet-18[16] model on some synthesized data, but the trained model produced poorly binarized images. Next, we performed a more straightforward histogram analysis of the MAN images to see if there was a global binarization threshold across the images. Histogram analysis of an image involves visually inspecting the distribution of grayscaled pixel intensities.

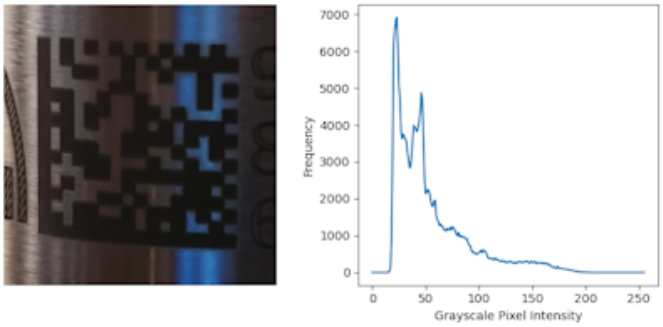
For an image binarization task, if an image contains a separation in peak grayscale pixel intensities, then binarizing the image with a threshold between the peaks will effectively separate the two areas in the image. Figure 7a highlights this calculated threshold, where there is a clear separation between two peaks of grayscale pixel intensities. In contrast, we see in Figure 7b a more minor separation between the peaks due to the poor lighting conditions, resulting in the failed binarization seen in 7c.

Unfortunately, while many images contain clear separations of grayscale value peaks in their histograms, there are still many failure cases similar to Figure 7b. These failure cases occur when there are light reflections and other related distortions within the area of a DMC. For these cases, no global threshold will produce a successful binarization. Because of this, other similar methods for automatic threshold selection, such as Otsu's method[11], will not work either. Therefore, a more complex method for effective binarization is required.

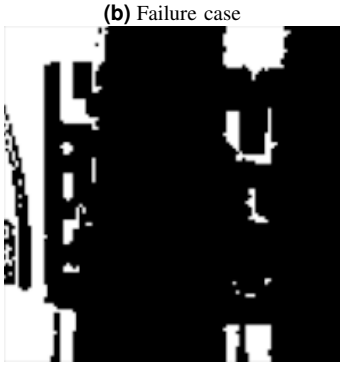
U-Net is an image segmentation model architecture developed for use in biomedical images. However, the application



(a) Success case



(b) Failure case



(c) Failure case when binarized with calculated threshold

Fig. 7: Binarization with histogram analysis. Note the more distinct separation of peaks in 7a compared to 7b.

of U-Net is not only limited to the medical industry, and its use cases are continually expanding. Since U-Nets development, the architecture has been used in image segmentation tasks for disease detection in leaves[17], [18], aerial image analysis for urban planning[19], and more. Because of its usage outside of medical applications, it stood to reason that it could be used to segment DMCs.

To train the U-Net model, we define that the model should, given an input image containing a DMC:

- Transform all pixels of the DMC completely black (0). These pixels would be present if the DMC were created with standard grid squares and without imperfections.
- Transform all pixels not of the DMC completely white (1). These are pixels belonging to the background surface the DMC is marked on or imperfections resulting from blur, which may extend the edges of the DMC grid squares.

The model should deal with all "noise" related to coloring, e.g., the DMC area not being perfectly black due to rust, oil, blur, light reflections, or other factors. The model should not deal with noise related to the shape of the DMC, such as rotation and warping.

To achieve this, a sigmoid layer followed by a binarization with a threshold of 0.5 was added as a head to the U-Net architecture. Figure 8 shows the complete architecture of the U-Net model used.

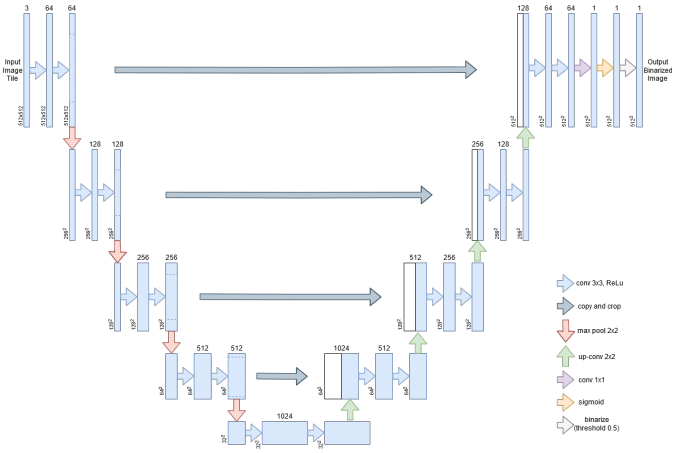


Fig. 8: Modified U-Net Architecture. The architecture differs from the original U-Net in the extra sigmoid and binarization layers at the end, and a slightly different padding method was used in the convolutional layers.

With the model well defined, the requirements of the dataset can be defined. Keeping in mind that the U-Net model will take the output of a YOLO model as input, the ideal input images for training are defined as the following:

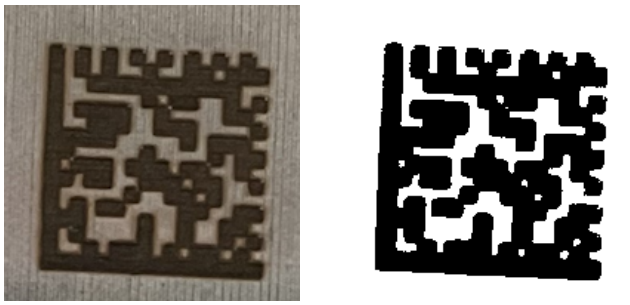
- The DMC within the image must cover a large area to simulate a cropped image from the YOLO model.
- The background of the DMC must be metallic, as Figure 2b shows this is the dominant background in the MAN-supplied images.
- The DMC *shape* should vary in rotation, shear, affine transformation, and perspective warping to simulate the examples seen in the MAN dataset.
- Due to the high amounts of blur distortion seen in Figure 2c, a random amount of blur should be applied to the images.
- Various random *color* variations (especially lighting distortion/noise) should be added to encourage model generalization. The model should learn to handle the main issue that globally thresholded binarization methods fail to handle.

Ground truth images are required for the model to learn how to binarize these images. The goal is to have the model binarize the images without rectification, so we define the ground truth images as counterparts to the training images in the following ways:

- Ground truth images should contain the same shape transformed DMC as its training image counterpart.
- Ground truth images are binarized images, where the black pixel values of the DMC are 0 ([0, 0, 0] in RGB

terms), and white background pixel values (in and around the DMC) are 1 ([255, 255, 255] in RGB terms).

An example of an ideal training and ground truth image pair is shown in Figure 9.



(a) Ideal "noisy" image **(b)** Ideal "ground truth" image

Fig. 9: Example of ideal noisy image and a (close to) ideal ground truth counterpart. Edited from a MAN-supplied image. More adverse lighting than seen here should be present in the synthetic images.

Because no public dataset like this existed, one needed to be synthesized. The synthesis process used libdmtx for DMC generation and PyTorch transforms v2[20] for applying the appropriate shape and color transformations. Metal textures were sourced from various free online sources and randomly cropped for use as backgrounds for the DMCs.

One thousand unique DMCs were generated, and a dataloader for augmenting the dataset appropriately during training was created. Figure 10 shows the synthesis process with an example training and ground truth image pair generated during the training process.

Of the 1000 generated DMCs, 900 were used for training, and the other 100 were used as a validation set. During training, on completion of each epoch, a model was evaluated on the validation set, and its loss was calculated. If the reduction in validation loss from the previous epoch was below a specified amount, the training would stop, and the model trained from the previous epoch would be saved. This form of early stopping helped reduce the model's overfitting to the synthetic training data.

We use the Python library "pillow" to load and compare images. This library also allows us to represent them as grayscale images with pixel values ranging from 0 (black) to 1 (white), the same range produced by the U-Net model architecture with its sigmoid layer head.

We trained three different U-Net binarizers, each optimizing for different loss functions. One for binary cross-entropy (BCE) loss, one for dice loss, and a final for an average of the two (BCEDice loss). In this case, BCE loss is calculated as the average of the differences between the predicted and ground-truth pixel values 11a. Dice loss here is calculated as the intersection between the predicted and ground-truth pixel values 11b. Theoretically, both can work as practical loss functions for this image binarization task, but to see which method (or a mix of the two 11c) produces the best model, we try all three options. Each model was trained and saved for later evaluation.

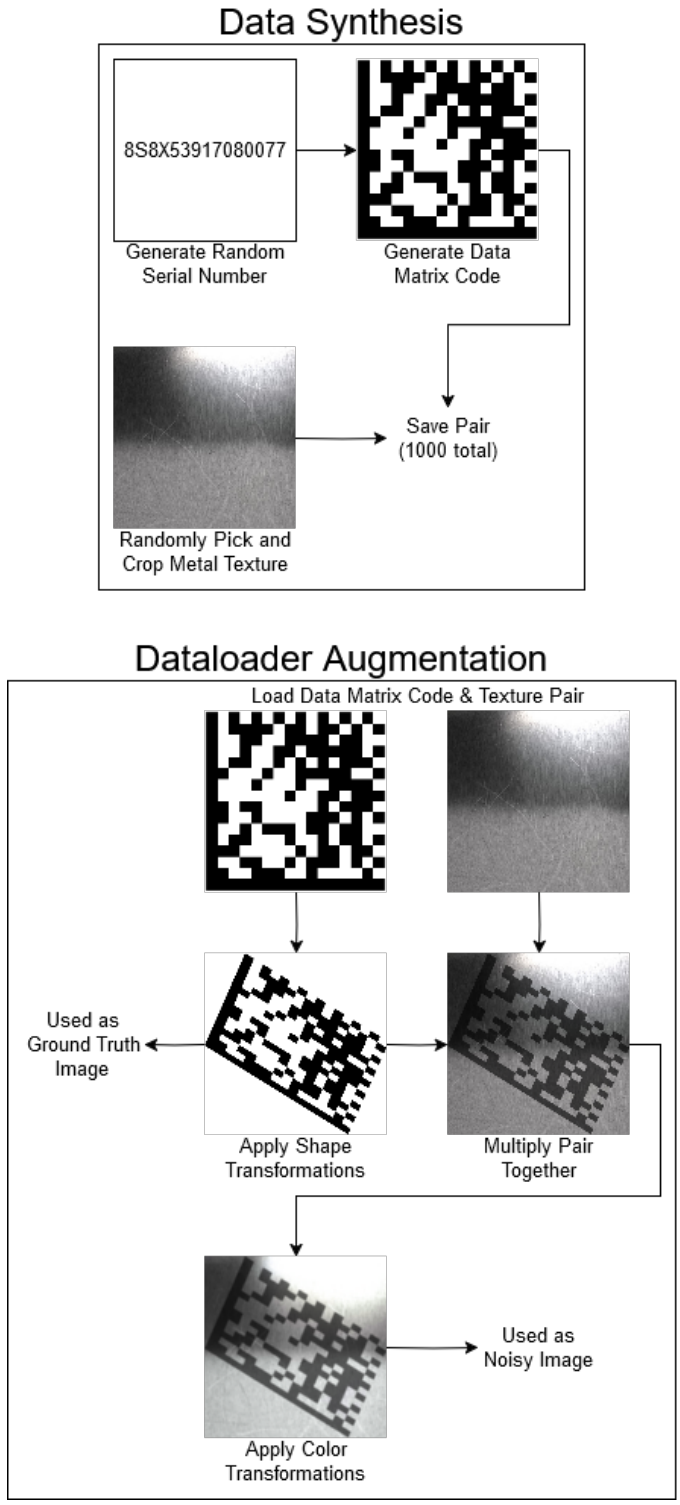


Fig. 10: The data synthesis and dataloader augmentation process. Note that while each epoch uses the same image pairs from the data synthesis, there is randomness introduced by the shape and color transformations.

$$BCELoss(x, y) = \frac{\{l_1, ..., l_N\}^\top}{N},$$
$$l_n = [y_n \cdot \log(x_n) + (1 - y_n) \cdot \log(1 - x_n)]$$

(a) BCE loss formula. $BCELoss(x, y)$ is the average batch loss comparing predicted images x with their ground truths y across all N images in a batch (batch size $N = 4$ in our case). l_n is the loss for a single n-th sample.

$$DiceLoss(x, y) = 1 - \frac{(2 \cdot \sum_{n=1}^{N} x_n y_n + \epsilon)}{(\sum_{n=1}^{N} x_n + \sum_{n=1}^{N} y_n + \epsilon)}$$

(b) Dice loss formula. $DiceLoss(x, y)$ is the average batch loss comparing predicted images x with their ground truths y across all N images in a batch (batch size $N = 4$ in our case). ϵ is the smoothing factor (standard of 1 in our case) to avoid $0/0$ cases.

$$BCEDiceLoss(x, y) = BCELoss_N + DiceLoss_N$$

(c) BCEDice loss formula. $BCEDiceLoss(x, y)$ is the summation of $BCELoss$ and $DiceLoss$ for batch N.

Fig. 11: Loss functions used to produce three differently optimized U-Net binarizers.

IV. Results & Discussion

A. DMC Localization Results

The YOLO models were each evaluated on the MAN test set along with the baseline libdmtx decoder. The full evaluation pipeline for each YOLO model was done as follows. For each image:

1) Use the given YOLO model to predict bounding boxes for possible DMC locations in the image.
2) Crop the image down to the dimensions of the highest likelihood bounding box.
3) Add 45 pixels of padding around the crop to ensure the model prediction does not accidentally cut off the DMC.
4) Decode with baseline decoder.

Table I shows how each model performed on the test set. Figure 12 shows all successfully decoded test image crops produced by the models.

Measure	Baseline	Kaggle Scratch	Kaggle Finetuned	Ultralytics Finetuned
Precision	-	0.25	0.91	**0.96**
Recall	-	0.24	0.84	**0.89**
F1	-	0.25	0.87	**0.92**
mAP50-95	-	0.07	0.75	**0.75**
DMC decode rate	**0.12**	0.04	**0.12**	**0.12**
FPS	1.07	**2.56**	2.02	1.96

TABLE I: Performance Comparison of YOLO Models on MAN test set. FPS is calculated on a laptop and should not be read directly; it is used to compare the speeds between methods.

All YOLO models reduce the overall runtime of the decoding pipeline. However, note that every subsequent YOLO model after the Kaggle from scratch model performs slower than the previous. Both fine-tuned models have the same decode rate as the baseline.

(a) Baseline **(b)** Kaggle Finetuned **(c)** Ultralytics Finetuned

Fig. 12: Crops from successful decoding from baseline decoder and YOLO model outputs. Note that Kaggle Scratch failed to detect and crop to this example.

B. DMC Localization Discussion

Comparing the resulting cropped images with the baseline, it becomes clear why the overall decoder speed increases even with an extra preprocessing step. The average sizes of the cropped images are much smaller than the baseline, meaning the decoder pipeline has fewer pixels to process.

However, cropping the images does not increase the decode rate of the baseline decoder. This is likely because cropping the image only improves step 2 (localizing the DMC) of the libdmtx pipeline, and the decode rate of libdmtx may not depend on the DMC size but rather the DMC's quality. The Kaggle from scratch model decreases the decode rate due to the YOLO model failing to detect and crop to some of the DMCs, and as a result, it is not used in further comparisons. The slower decode rate of the finetuned models is likely due to a higher number of false positives in predicted bounding boxes compared to the Kaggle from scratch model.

The Ultralytics Finetuned model performs best in all measures except speed and is therefore selected as the YOLO model to use in step 2 with the U-Net binarizer.

Overall, we consider step 1 a success, as performing DMC localization with yolo11n has successfully reduced the baseline decoder's overall decode time without sacrificing the rate of decoding.

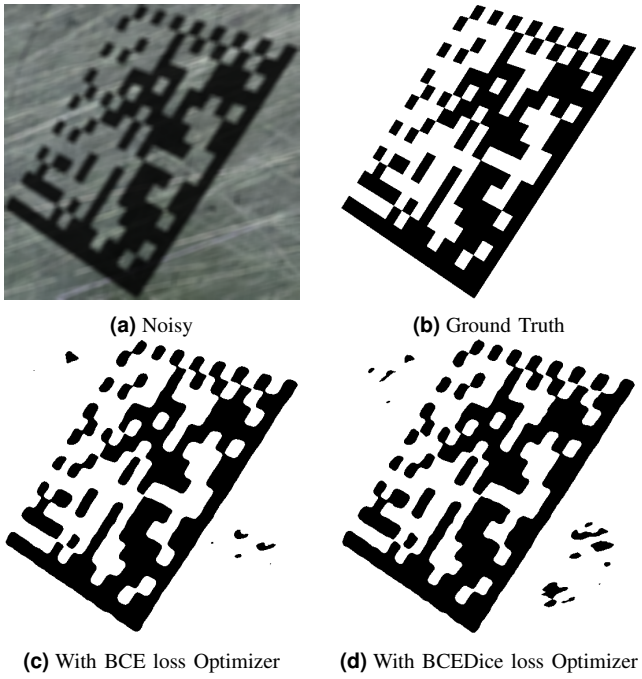
C. DMC Binarizer Results

Each trained U-Net binarizer was evaluated on the validation set and YOLO croppings of all MAN-supplied images containing laser markings (58 images). Because the binarizers were not finetuned to the real-world MAN data, bias is only present in the YOLO cropping step and not in the binarizer step.

A sample of the different model binarizations can be seen below in Figure 13. The model optimized purely for Dice loss produced only white images on the validation set and was therefore excluded from all evaluations.

The models trained for BCE loss and BCEDice loss produced similarly binarized images. The results of the two models on all MAN images containing laser-marked DMCs, along with samples of the model binarizations, are shown in Table II and Figure 14 below.

Both models sacrifice overall runtime to increase the decode rate of the baseline decoder. The model optimizing for both BCE and Dice loss outperforms the BCE loss model at an overall decode rate of $\sim 55\%$.



(a) Noisy **(b)** Ground Truth

(c) With BCE loss Optimizer **(d)** With BCEDice loss Optimizer

Fig. 13: Sample of Validation Binarizations with two successful models.

Measure	Baseline	BCE Loss	BCEDice Loss
DMC decode rate	0.43	0.48	**0.55**
FPS	1.23	0.32	0.32

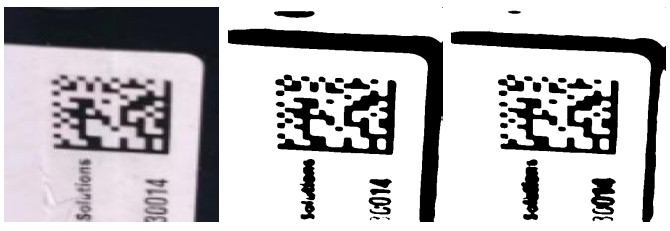
TABLE II: Performance Comparison of U-Net Binarizer Models on all MAN images containing laser-marked DMCs.

D. DMC Binarizer Discussion

Despite the failure of the model optimizing purely for Dice loss, the model optimizing for both BCE and Dice loss performs the best in practice. The model optimizing for Dice loss may have failed due to the lack of class balancing between black and white pixels. We could likely reduce the class imbalance problem by weighing the classes evenly across batches, potentially utilizing dice loss better. However, the dice loss combined with BCE loss leads to an even better decode rate than only optimizing for BCE loss.

Overall, the U-Net architecture can effectively be utilized for binarization tasks, as seen by the improvement in decode rate. However, the increase in runtime results in a decoding pipeline that runs in less than real-time, taking an average of $3s$ to process each image.

The model's high runtime is partially due to the high dimensionality of the input and output images. The size of the input images is three channels (RGB) 512x512, while the output is one channel (Grayscale) 512x512 images. It should be explored if having the input grayscale and scaling down the size could increase the model's forward pass speed without sacrificing the decode rate. A good first step for experimenting with this could be replicating the original U-Net architecture more closely by inputting grayscale images of sizes 572x572. Still, depending on how vital the color information was for achieving good binarization, it may produce lower decode rates.



(a) Input Image **(b)** BCE Loss **(c)** BCEDice Loss

Fig. 14: Comparison of Binarization between optimizer methods.

V. Final Discussion & Conclusion

We have shown that machine learning methods can effectively be utilized to improve the decode rate of standard scanners. However, with the techniques used in this paper, the enhanced decode rate comes at the cost of increased runtime. With more research into this area, it may be possible to increase the decode rate further and optimize the methods used to reduce the runtime for real-time usage.

A. Localization with YOLO

Localizing the DMC area with a finetuned version of Ultralytics pretrained yolo11n model successfully speeds up the baseline decoder without sacrificing its decode rate.

B. Binarization with U-Net

Using U-Net to segment the DMCs from the rest of the images via image binarization successfully improved the decode rate of the baseline decoder. However, the U-Net model architecture proved more computationally expensive than the baseline decoder, leading to a slower than real-time processing speed of around $3s$.

C. Limitations

The most significant limitation of this project was access to real-world data. It would be best to have thousands of real-world images of the DMCs marked on metal components in various conditions. However, obtaining these images requires requesting workers from across the globe to take them, and they would need to be manually annotated to get their YOLO bounding boxes and U-Net ground-truth labels. While technically possible, this process would be expensive and require backing from MAN.

Another limitation was time. More time to develop and optimize the methods, particularly the U-Net architecture, could improve the models' final runtime.

D. Future Work

Research into rectification of the DMCs, perhaps after the binarization step, could result in a decoding pipeline able to generalize for differently shaped DMCs, such as the dot-peen marked DMCs.

Furthermore, the U-Net binarization speed could be improved by reducing the dimensionality of the input and output

images. Depending on how relevant the color information was for binarizing the images, the inputs could be significantly reduced by providing grayscale images and inputs instead of RGB.

Experimentation with balancing the class weights during Dice loss calculation could lead to an improved BCEDice loss optimizer.

More versions of yolo11n exist that could be utilized as alternatives to the models used in this paper. For example, yolo11n-pose is a YOLO object detection model that predicts oriented bounding boxes, which could improve the decode rate of libdmtx even further by effectively performing part of step 3 (Geometric Alignment). Alternatively to U-Net, there also exists yolo11n-seg, an Ultralytics YOLO model designed for segmentation tasks. yollo11n-seg may outperform U-Net's speed and performance.

It was seen through earlier experimentation that care was necessary when defining the steps for data synthesis. For example, some previously used metal textures were too different from the real-world examples, leading to training models that did not generalize well to the MAN-supplied data. A more thorough investigation into which data synthesis methods produce good models for this domain could lead to a better understanding of relevant factors to synthesize, possibly producing better-trained models. The data synthesis could even be taken a step further by simulating the data in 3 dimensions. Three-dimensional CAD models (Computer-aided design) available from MAN could also be used, with randomized parameters to simulate pointing a phone camera at a DMC pasted onto it.

ACKNOWLEDGMENT

The author would like to thank MAN Energy Solutions for providing the dataset of real-world images containing Data Matrix codes, and their supervisors, Yucheng Lu & Veronika Cheplygina from the IT University of Copenhagen research group DASYA for guiding the entire process.

REFERENCES

[1] M. Laughton and M. Straight, *Datamatrix reading and writing library, utils and wrappers*. [Online]. Available: https://github.com/dmtx.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, *You only look once: Unified, real-time object detection*, 2016. arXiv: 1506.02640 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1506.02640.

[3] O. Ronneberger, P. Fischer, and T. Brox, *U-net: Convolutional networks for biomedical image segmentation*, 2015. arXiv: 1505.04597 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1505.04597.

[4] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023, ISSN: 2504-4990. DOI: 10.3390/make5040083. [Online]. Available: https://www.mdpi.com/2504-4990/5/4/83.

[5] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using yolo: Challenges, architectural successors, datasets and applications," *multimedia Tools and Applications*, vol. 82, no. 6, pp. 9243–9275, 2023.

[6] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, "A review of yolo algorithm developments," *Procedia computer science*, vol. 199, pp. 1066–1073, 2022.

[7] G. Jocher and J. Qiu, *Ultralytics yolo11*, version 11.0.0, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics.

[8] R. Chen, Y. Yu, X. Xu, L. Wang, H. Zhao, and H.-Z. Tan, "Adaptive binarization of qr code images for fast automatic sorting in warehouse systems," *Sensors*, vol. 19, no. 24, p. 5466, 2019.

[9] R. Chen, W. Li, K. Lan, J. Xiao, L. Wang, and X. Lu, "Fast adaptive binarization of qr code images for automatic sorting in logistics systems," *Electronics*, vol. 12, no. 2, p. 286, 2023.

[10] P. V. Bezmaternykh, D. A. Ilin, and D. P. Nikolaev, "U-net-bin: Hacking the document image binarization contest,", vol. 43, no. 5, pp. 825–832, 2019.

[11] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. DOI: 10.1109/TSMC.1979.4310076.

[12] *Everything you need to build and deploy computer vision applications*, 2024. [Online]. Available: https://roboflow.com.

[13] T.-Y. Lin, M. Maire, S. Belongie, *et al.*, *Microsoft coco: Common objects in context*, 2015. arXiv: 1405.0312 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1405.0312.

[14] Hamid, *Yolo-qr-labeled*, version 1, 2024. [Online]. Available: https://www.kaggle.com/datasets/hamidl/yoloqrlabeled.

[15] L. Karrach and E. Pivarciova, "Comparative study of data matrix codes localization and recognition methods," *Journal of Imaging*, vol. 7, p. 163, Aug. 2021. DOI: 10.3390/jimaging7090163.

[16] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, 2015. arXiv: 1512.03385 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1512.03385.

[17] M. E. Chowdhury, T. Rahman, A. Khandakar, *et al.*, "Automatic and reliable leaf disease detection using deep learning techniques," *AgriEngineering*, vol. 3, no. 2, pp. 294–312, 2021.

[18] S. Zhang and C. Zhang, "Modified u-net for plant diseased leaf image segmentation," *Computers and Electronics in Agriculture*, vol. 204, p. 107 511, 2023.

[19] Z. Pan, J. Xu, Y. Guo, Y. Hu, and G. Wang, "Deep learning segmentation and classification for urban village using a worldview satellite image based on u-net," *Remote Sensing*, vol. 12, no. 10, p. 1574, 2020.

[20] A. Paszke, S. Gross, F. Massa, *et al.*, *Pytorch transforms v2 documentation*, Accessed: 2024-12-14, 2024. [Online]. Available: https://pytorch.org/vision/0.20/transforms.html.